

Smart Contract Security Audit Report

TSwap

Audit Date: May 15, 2026

Auditor: BlockChainBabbie

Commit Hash: e643a8d4c2c802490976b538dd009b351b1c8dda

Executive Summary

Total Issues Found:

Critical:

High: 3

Medium: 3

Low: 1

Informational/Gas: 3

Overall Risk Rating: High

Status: 10 Fixed | 0 Acknowledged | 0 In Progress

Scope

Contracts in Scope:

Contract	File Path	Lines of Code
PoolFactory.sol	src/PoolFactory.sol	35
TSwapPool.sol	src/TSwapPool.sol	169
...	...	...

Audit Methodology

- Thorough manual code review
- Static analysis (Slither)
- Fuzzing & invariant testing (Foundry)

Findings Summary

Severity	Count	Fixed	Acknowledged
Critical	0	0	0
High	3	3	0
Medium	3	3	0
Low	1	1	0
Informational	3	3	0

Detailed Findings

Highs

[H-01] Incorrect calculation of fees in a `TSwapPool::getInputAmountBasedOnOutput`

Description

The function `TSwapPool::getInputAmountBasedOnOutput` does not calculate the fee(0.03%) correctly because of the incorrect ratio in the function

Impact

A user who calls the `TSwapPool::getInputAmountBasedOnOutput` or any function that uses this function

will be returned an incorrect `inputAmount`

Mitigation

```
function getInputAmountBasedOnOutput(
    uint256 outputAmount,
    uint256 inputReserves,
    uint256 outputReserves
)
    public
    pure
    revertIfZero(outputAmount)
    revertIfZero(outputReserves)
    returns (uint256 inputAmount)
{
-     return ((inputReserves * outputAmount) * 10000) / ((outputReserves - outputAmount) * 99;
+     return ((inputReserves * outputAmount) * 1000) / ((outputReserves - outputAmount) * 99;
}
```

The function uses 10000/997 instead of a 1000/997 thereby calculating wrong fees

[H-02] The function `TSwapPool1::swapExactOutput` missing MEV attack protection/ Slippage attacks

Description

The function `TSwapPool1::swapExactOutput` has the premise for an MEV attack and in the scenario where a user can then approve a transaction that will then return or swap `outputAmount` that might be in-significant, leading the user to lose money

Impact

The impact of the bug will affect a user that decides to swap token when the pool reserve of the poolToken and wrappedETH is unfavourable. Since we don't have a slippage check, and MEV bots might decide to front the transaction making a big change in the pool then we get unfavourable conditions, for the swap and without a slippage check said will lose money.

Mitigation

```

function swapExactOutput(
    IERC20 inputToken,
    IERC20 outputToken,
+   uint256 minInputAmount
    uint256 outputAmount,
    uint64 deadline
)
    public
    revertIfZero(outputAmount)
    revertIfDeadlinePassed(deadline)
    returns (uint256 inputAmount)
{
    uint256 inputReserves = inputToken.balanceOf(address(this));
    uint256 outputReserves = outputToken.balanceOf(address(this));

+   if (outputAmount < minInputAmount) {
+       revert TSwapPool1__InputTooLow(outputAmount, minOutputAmount);
+   }

    inputAmount = getInputAmountBasedOnOutput(outputAmount, inputReserves, outputReserves);

    _swap(inputToken, inputAmount, outputToken, outputAmount);
}

```

[H-03] The core invariant breaks in the TSwapPool1::_swap()

Description

The core invariant if the contract which is $x * y = k$ is broken the the internal function `_swap` is called more than 10 times

Impact

This gives an avenue to an attacker to continuously call a the swap function will measly swaps more than 10 then get an incentive.

An attacker can call this multiple times until the poolToken and wETH reserve is all drained

Proof of Concept

```

function test_invariantX() public {
    vm.startPrank(liquidityProvider);
    weth.approve(address(pool), 100e18);
    poolToken.approve(address(pool), 100e18);
    pool.deposit(100e18, 100e18, 100e18, uint64(block.timestamp));
    int256 startingX = int256(weth.balanceOf(address(pool)));
    int256 startingY = int256(poolToken.balanceOf(address(pool)));

    uint256 outputAmount = 10e17;
    uint256 poolTokenAmount = pool.getInputAmountBasedOnOutput(
        outputAmount, poolToken.balanceOf(address(pool)), weth.balanceOf(address(pool))
    );

    int256 expectedDeltaX = int256(outputAmount) * -1;
    int256 expectedDeltaY = int256(poolTokenAmount);

    vm.startPrank(user);
    poolToken.approve(address(pool), type(uint256).max);
    pool.swapExactOutput(IERC20(poolToken), IERC20(weth), outputAmount, uint64(block.timestamp));
    pool.swapExactOutput(IERC20(poolToken), IERC20(weth), outputAmount, uint64(block.timestamp));
    pool.swapExactOutput(IERC20(poolToken), IERC20(weth), outputAmount, uint64(block.timestamp));
    pool.swapExactOutput(IERC20(poolToken), IERC20(weth), outputAmount, uint64(block.timestamp));
    pool.swapExactOutput(IERC20(poolToken), IERC20(weth), outputAmount, uint64(block.timestamp));
    pool.swapExactOutput(IERC20(poolToken), IERC20(weth), outputAmount, uint64(block.timestamp));
    pool.swapExactOutput(IERC20(poolToken), IERC20(weth), outputAmount, uint64(block.timestamp));
    pool.swapExactOutput(IERC20(poolToken), IERC20(weth), outputAmount, uint64(block.timestamp));
    pool.swapExactOutput(IERC20(poolToken), IERC20(weth), outputAmount, uint64(block.timestamp));
    pool.swapExactOutput(IERC20(poolToken), IERC20(weth), outputAmount, uint64(block.timestamp));
    vm.stopPrank();

    int256 endingX = int256(weth.balanceOf(address(pool)));
    int256 endingY = int256(poolToken.balanceOf(address(pool)));

    int256 actualDeltaX = endingX - startingX;
    int256 actualDeltaY = endingY - startingY;

    console.log("actualDeltaX :", actualDeltaX );
    console.log("expectedDeltaX :", expectedDeltaX);

    assertEq(actualDeltaX, expectedDeltaX);

```

The return is this

```
[FAIL: assertion failed: -11000000000000000000 != -1000000000000000000] test_invariantX() (gas:
```

Mitigations

```
function _swap(IERC20 inputToken, uint256 inputAmount, IERC20 outputToken, uint256 outputAmount)
    if (_isUnknown(inputToken) || _isUnknown(outputToken) || inputToken == outputToken) {
        revert TSwapPool__InvalidToken();
    }

    swap_count++;
-   if (swap_count >= SWAP_COUNT_MAX) {
-       swap_count = 0;
-       outputToken.safeTransfer(msg.sender, 1_000_000_000_000_000_000);
-   }

    emit Swap(msg.sender, inputToken, inputAmount, outputToken, outputAmount);

    inputToken.safeTransferFrom(msg.sender, address(this), inputAmount);
    outputToken.safeTransfer(msg.sender, outputAmount);
}
```

Mediums

[M-01] The parameter deadline is not used in the TSwapPool::deposit as it should be

Description

The deposit function takes a parameter called the `deadline` that, it is supposed from the documation '@param deadline The deadline for the transaction to be completed by'

Impact

The absence of the deadline checks allows the depositor, to deposit even if the deadline has been passed.

Mitigation

```

function deposit(
    uint256 wethToDeposit,
    uint256 minimumLiquidityTokensToMint,
    uint256 maximumPoolTokensToDeposit,
    uint64 deadline
)
    external
    revertIfZero(wethToDeposit)
+   revertIfDeadlinePassed(deadline)
    returns (uint256 liquidityTokensToMint)

```

[M-02] The event in the `_addLiquidityMintAndTransfer` called event `LiquidityAdded` returns false information

Description

In this function `TSwapPool::_addLiquidityMintAndTransfer` it emits an event that does not pass the right information.

Impact

Wrong information passed to the protocol reader and people who will read the event

Mitigation

```

_mint(msg.sender, liquidityTokensToMint);
- emit LiquidityAdded(msg.sender, poolTokensToDeposit, wethToDeposit);
+ emit LiquidityAdded(msg.sender, wethToDeposit, poolTokensToDeposit);

```

[M-03] The function `TSwapPool::sellPoolTokens` does not use the right function to swap token.

Description

The function `sellPoolToken` sole purpose is to allow user sell poolToken in place of wETH and this function does not enable that functionality

Impact

The function `sellPoolToken` use the `swapExactOutput` instead of the `swapExactInput` that will hinder the user from using the `sellPoolToken` the way it is supposed to be used.

Because in the documentation we have this for the `swapExactOutput` "You say "I want 10 output

WETH, and my input is DAI"

* The function will figure out how much DAI you need to input to get 10 WETH

* And then execute the swap"

That is not the intended functionality of the `sellPoolToken` function.

Mitigation

```
function sellPoolTokens(uint256 poolTokenAmount, uint256 minOutputAmount ) external returns (uint256)
{
    return
    -     swapExactOutput(i_poolToken, i_wethToken, poolTokenAmount, uint64(block.timestamp));
    +     swapExactInput(i_poolToken, poolTokenAmount, i_wethToken, minOutputAmount, uint64(block.timestamp));
}
```

Lows

[L-01] The `TSwapPool::swapExactInput` returns an output function, however the function never returns it.

Description

In the function `TSwapPool::swapExactInput` the return function is not properly initialized.

Impact

A user will get an improper return value and it will be misleading information

Mitigations


```

function swapExactInput(
    IERC20 inputToken,
    uint256 inputAmount,
    IERC20 outputToken,
    uint256 minOutputAmount,
    uint64 deadline
)
    public
    revertIfZero(inputAmount)
    revertIfDeadlinePassed(deadline)
    //not returnedddd
    returns (uint256 output)
{
    uint256 inputReserves = inputToken.balanceOf(address(this));
    uint256 outputReserves = outputToken.balanceOf(address(this));

-     uint256 outputAmount = getOutputAmountBasedOnInput(inputAmount, inputReserves, outputReserves);
+     uint256 output = getOutputAmountBasedOnInput(inputAmount, inputReserves, outputReserves);

-     if (outputAmount < minOutputAmount) {
-         revert TSwapPool__OutputTooLow(outputAmount, minOutputAmount);
-     }
+     if (output < minOutputAmount) {
+         revert TSwapPool__OutputTooLow(output, minOutputAmount);
+     }

-     _swap(inputToken, inputAmount, outputToken, outputAmount);
+     _swap(inputToken, inputAmount, outputToken, output);
}

```

Or an alternative will be

```

function swapExactInput(
    IERC20 inputToken,
    uint256 inputAmount,
    IERC20 outputToken,
    uint256 minOutputAmount,
    uint64 deadline
)
    public
    revertIfZero(inputAmount)
    revertIfDeadlinePassed(deadline)
    //not returnedddd
-     returns (uint256 output)
+     returns (uint256 outputAmount)
{
    uint256 inputReserves = inputToken.balanceOf(address(this));
    uint256 outputReserves = outputToken.balanceOf(address(this));

    uint256 outputAmount = getOutputAmountBasedOnInput(inputAmount, inputReserves, outputRe:

    if (outputAmount < minOutputAmount) {
        revert TSwapPool__OutputTooLow(outputAmount, minOutputAmount);
    }

    _swap(inputToken, inputAmount, outputToken, outputAmount);
}

```

Informational/Gas

[I-01] Un-used error in PoolFactory.sol

Description

PoolFactory has an un-used error `errorPoolFactory__PoolDoesNotExist(address tokenAddress);`

Impact

This `errorPoolFactory__PoolDoesNotExist(address tokenAddress);` is not used anywhere in the contract `PoolFactory.sol` so therefore a wasted of gas.

Mitigations

```
error PoolFactory__PoolAlreadyExists(address tokenAddress);  
- error PoolFactory__PoolDoesNotExist(address tokenAddress);
```

Removing the un-used error.

[I-02] No zero checks for address(0) in constructor in PoolFactory.sol

Description

No zero checks in place in the scenario where a address(0) is passed in the constructor.

Impact

In a scenario where a zero address is passed through, we then create a pool that doesn't have a functioning `i_wethToken`, leading to a creation of a dormant pool contract.

Mitigations

```
constructor(address wethToken) {  
+   if( wethToken == address(0)) {  
+       revert();  
+}  
    i_wethToken = wethToken;  
}
```

[I-03] No use for poolTokenReserves in the TSwapPool::deposit

Description

The `uint256 poolTokenReserves = i_poolToken.balanceOf(address(this));` is not used anywhere in the function, therefore a waste of gas

Mitigation

```
if (totalLiquidityTokenSupply() > 0) {  
    uint256 wethReserves = i_wethToken.balanceOf(address(this));  
  
-    uint256 poolTokenReserves = i_poolToken.balanceOf(address(this));}
```

Summary

This report represents an early-stage independent security review conducted as part of my practical learning journey in smart contract security and auditing. The focus of this review was to improve my understanding of protocol design, invariant testing, MEV considerations, and vulnerability discovery through hands-on analysis.

The findings included in this report were identified through a combination of:

- manual code review

- static analysis

- fuzzing

- invariant testing using Foundry

This work was carried out while following the Cyfrin Updraft curriculum, with guidance and walkthrough support from Patrick Collins, which helped shape my approach to testing and protocol reasoning.